

TriArchitect: A Shared-State Three-Agent Architecture for Safe Large-Scale Legacy Codebase Migration

Neeraj Kumar Singh Beshane

Meta / Facebook, USA



Abstract

Enterprises face significant challenges migrating legacy systems due to technical debt accumulation and dependency complexity that exceeds current automated migration capabilities. Single-shot Large Language Model tools fail on interconnected enterprise architectures while sequential and parallel multi-agent systems suffer from context decay and consistency problems, respectively. This article introduces TriArchitect, a shared-state multi-agent framework designed to address the dependency hallucination problem in automated migration through continuous collaboration between three specialized agents. The Archeologist agent performs semantic extraction and business rule identification, the Architect agent generates cloud-native target designs under constraints, and the Validator agent provides continuous equivalence verification. A Typed Migration Graph maintains a dynamic ontological representation of code, configuration, and build artifacts while enabling real-time state synchronization across agents. The Cyclic Consensus Protocol ensures comprehensive validation through signed migration verdicts and pre-merge verification gates. Production deployment across interconnected services supporting consumer electronics products demonstrates substantial improvements in migration efficiency, operational cost reduction, compute resource reclamation, development cycle acceleration, and build accuracy while maintaining zero production incidents throughout the transformation process.

Keywords: Multi-Agent Systems, Software Migration, Legacy Modernization, Automated Transformation, Enterprise Architecture

1. Introduction

Market Context and Scale

The global enterprise software landscape faces a critical modernization challenge. Legacy systems dominate organizational infrastructure while hindering technological advancement. Application modernization services represent a rapidly expanding market segment. Organizations worldwide recognize the urgent need for systematic modernization approaches. Cloud migration initiatives drive substantial market demand across industries. Digital transformation requirements accelerate modernization spending patterns. The market demonstrates consistent growth trajectories across geographical regions. Enterprise adoption of cloud-native architectures fuels service demand. Modernization complexity increases with organizational scale and system interdependencies. Service providers develop specialized offerings for different industry verticals [1].

Enterprise technical debt accumulates through decades of incremental system development. Maintaining obsolete technological infrastructure incurs significant expenses for businesses. Compared to their corporate value contribution, legacy systems use excessive resources. Technical debt shows itself as slower development velocity and greater maintenance expenditure. As companies increase their digital footprints, system complexity rises exponentially. Linking legacy systems with contemporary platforms increases integration challenges. Performance constraints of earlier technologies restrict company agility and competitiveness. Security weaknesses in legacy systems present growing hazards to corporate activities. Old technologies make it harder to meet compliance rules. Modernizing drive grows stronger as digital rivalry quickens throughout sectors [1].

Legacy systems continue to be essential infrastructure elements in several industries. Financial organizations depend mostly on systems created decades ago. Government departments run mission-critical operations on obsolete platforms. Healthcare organizations maintain patient systems on legacy technologies. Manufacturing companies depend on industrial control systems with limited modernization options. These systems contain irreplaceable business logic accumulated over extensive operational periods. Domain expertise embedded within legacy code represents significant organizational assets. Regulatory compliance adaptations reflect years of institutional knowledge and experience. Performance optimizations within existing systems often lack proper documentation or understanding [1].

Current Limitations of LLM-Based Migration Tools

Large Language Models provide promising capabilities for automated code transformation tasks. Current implementations treat migration as isolated text processing problems. These approaches ignore complex architectural dependencies that span multiple system components. Single-shot generation tools demonstrate effectiveness on individual code segments. Enterprise-scale systems require an understanding of interconnected service relationships and data flows. Context preservation becomes critical when transforming business logic across service boundaries. Existing tools lack mechanisms for maintaining semantic consistency during transformations. Integration points between services often contain implicit assumptions and behavioral contracts [2].

Sequential agent architectures attempt to address complexity through specialized processing stages. Information passes between agents through predetermined communication protocols. Context degradation occurs at each handoff point in the processing pipeline. Critical semantic information about business requirements gets lost during agent transitions. Performance constraints and security assumptions may not transfer accurately between processing stages. Multi-step approaches create dependencies between agent capabilities and coordination mechanisms. Error propagation becomes problematic when upstream agents

make incorrect assumptions. Sequential processing introduces bottlenecks that limit scalability for large codebases [2].

Independent parallel agent systems explore concurrent processing as an alternative approach. Multiple agents operate simultaneously across different system components without coordination. Consistency challenges emerge when agents make conflicting assumptions about shared interfaces. Integration testing reveals incompatibilities only after individual agent processing completes. Parallel approaches require sophisticated coordination mechanisms to prevent conflicting modifications. Agent specialization improves individual component handling but complicates system-wide consistency. Synchronization overhead can negate the performance benefits of parallel processing. Load balancing becomes complex when agents have different processing requirements and capabilities [2].

Research Problem and Objectives

Dependency hallucination represents a critical challenge in generative migration systems. Artificial intelligence tools generate plausible transformations that violate essential architectural constraints. These violations often remain undetected until runtime or under specific operational conditions. Current validation approaches operate as final pipeline steps rather than continuous processes. Early detection mechanisms could prevent problematic transformations from propagating through migration workflows. Continuous verification requires real-time collaboration between specialized analysis components. Behavioral equivalence testing becomes essential for maintaining system correctness during transformations [2].

Symbiotic multi-agent architecture offers potential solutions to coordination challenges. Real-time communication enables agents to maintain shared understanding throughout processing. Collaborative decision-making can prevent conflicts that arise in independent processing approaches. Consensus mechanisms ensure that transformations meet multiple validation criteria before implementation. Shared state management eliminates information loss that occurs in traditional handoff processes. Dynamic coordination allows agents to adapt their strategies based on emerging requirements or constraints [2].

Production validation demonstrates the practical viability of proposed architectural approaches. Enterprise-scale deployment scenarios provide comprehensive testing environments for automated migration systems. Safety and reliability standards must be maintained throughout large-scale transformation processes. Integration compatibility ensures that migrated systems continue operating within existing infrastructure. Performance characteristics require careful preservation during modernization activities. Operational metrics validate that automated approaches achieve intended business outcomes [2].

Key Contributions

Typed Migration Graph introduces novel approaches to shared state management during transformation processes. Dynamic knowledge structures capture semantic relationships between diverse system artifacts. Code components maintain their contextual relationships throughout the migration lifecycle. Configuration dependencies receive explicit representation within the shared state model. Build artifacts and their relationships become visible to all participating agents. Real-time consistency mechanisms ensure that all agents operate with identical system understanding. The graph structure enables precise reasoning about complex dependency patterns that span multiple architectural layers [2].

Cyclic Consensus Protocol provides systematic approaches to continuous verification during automated transformations. Multiple validation perspectives ensure comprehensive assessment of proposed changes. Verification activities are integrated throughout the transformation process rather than operating as final steps. Early detection capabilities identify potential issues before they propagate through migration

workflows. Consensus mechanisms prevent unsafe transformations from reaching production environments. The protocol maintains safety standards while enabling automated processing at enterprise scale [2].

Production validation across interconnected services demonstrates the practical viability of shared-state multi-agent architectures. Large-scale deployment scenarios provide comprehensive evaluation environments for automated migration technologies. Enterprise-grade safety and reliability standards guide system design and implementation decisions. Mission-critical business systems require specialized approaches that balance automation benefits with operational risk management. The validation encompasses technical correctness, operational compatibility, and business continuity requirements throughout the transformation process [2].

2. Related Work and Current Approaches

Evolution of Migration Approaches

Enterprise organizations encounter substantial transformation pressures as technological landscapes evolve rapidly. Legacy infrastructure modernization becomes essential for maintaining competitive advantages and operational efficiency. Historical migration practices depended primarily on manual translation processes and extensive system redesign activities. These traditional methods consumed excessive time and resources and introduced frequent errors in complex organizational environments. Contemporary enterprises need structured transformation methodologies that accommodate interconnected application networks and maintain essential business functionality. The modernization journey must eliminate accumulated technical debt while incorporating current architectural standards. Older systems frequently embed undocumented operational rules that constitute valuable institutional wisdom. Transformation methodologies must identify and retain this implicit knowledge during conversion activities. Enterprise transformation challenges encompass organizational dynamics alongside technical implementation requirements. Effective modernization demands collaboration between engineering teams and business leadership throughout the transformation lifecycle [3].

Enterprise application environments introduce distinct obstacles for automated migration technologies and transformation frameworks. Intricate interdependencies among application components demand sophisticated evaluation and strategic planning methodologies. Operational continuity demands impose restrictions on migration approaches and implementation schedules. Compliance requirements must remain satisfied throughout the complete transformation cycle. Security measures become more and more vital as sensitive data and mission-critical business processes are transferred. During modernization projects, as stakeholder expectations rise, performance expectations often increase. Knowledge transfer from legacy application environments becomes vital when original development personnel are unavailable. Documentation deficiencies create barriers to understanding current system operations and design choices. Migration strategies must accommodate different levels of technical debt across various application components. Integration validation becomes crucial for confirming that transformed applications meet existing operational standards [3].

Modern migration platforms utilize artificial intelligence capabilities for automated code evaluation and system transformation activities. Machine learning algorithms facilitate pattern identification within complex code repositories and architectural frameworks. Natural language processing techniques support business logic extraction from code annotations and technical documentation. Automated test case generation assists in validating functional equivalence between original and converted applications. These technical advancements allow transformation projects at scales never before possible by hand. Still,

company complexity usually goes beyond what existing automatic systems can handle; therefore needs human oversight and participation. Hybrid methodologies combining automated processing with specialist validation demonstrate encouraging outcomes in production implementations. The progression toward intelligent migration platforms continues as artificial intelligence technologies mature [4].

Existing Multi-Agent Architectures

Multi-agent architectural frameworks concentrate on collaborative problem resolution through distributed intelligent processing elements. Agent coordination protocols enable complex challenges to be distributed across specialized processing components. Communication frameworks support information exchange between agents during collaborative processing operations. Consensus mechanisms ensure uniform decision-making when multiple agents participate in solution development. Resource distribution strategies enhance utilization across agent networks for optimal performance characteristics. Fault tolerance protocols maintain operational reliability when individual agents experience processing failures or errors. Agent specialization facilitates expertise development within particular problem domains or processing functions. Dynamic reconfiguration enables agent networks to adjust to evolving requirements and processing environments [4].

Sequential processing frameworks implement predefined workflows where agents function in designated, ordered sequences. Individual agents receive information from preceding processing phases and generate output for following agents. Information movement follows established patterns that reduce complexity in agent coordination and communication activities. Error management becomes difficult when failures happen in upstream agents and impact downstream processing operations. Performance constraints develop when individual agents cannot handle information at the speeds required by the overall system capacity. Context maintenance demands careful design of information transfer protocols between sequential processing phases. Specialization advantages allow individual agents to optimize for particular transformation or evaluation tasks. Debugging and problem resolution become more challenging when errors spread through multiple processing phases [4].

Parallel agent frameworks attempt to resolve capacity limitations through simultaneous processing of independent system elements. Multiple agents function concurrently on different aspects of the overall challenge without requiring sequential coordination activities. Resource utilization is enhanced when agents can utilize available computational resources across a distributed processing infrastructure. Consistency obstacles emerge when agents make choices about shared resources or interconnected system elements. Synchronization protocols become necessary to prevent conflicts when agents modify overlapping data structures or system artifacts. Load distribution requires sophisticated scheduling algorithms to allocate work effectively across available processing agents. Integration validation reveals conflicts and inconsistencies only after individual agent processing completes successfully [4].

Gap Analysis

Enterprise digital transformation encounters substantial obstacles in managing complex system interdependencies and architectural configurations. Legacy applications frequently display circular dependency relationships that challenge traditional evaluation methodologies. Current migration tools experience difficulties with bidirectional communication configurations where services maintain mutual dependencies for operational functionality. Authentication and authorization frameworks regularly exhibit these complex interdependency configurations in production implementations. User management services rely on authentication elements, while authentication frameworks require user information for access control processes. Disrupting these circular relationships inappropriately can significantly modify intended system operations and weaken security frameworks. Current automated migration solutions lack

advanced reasoning capabilities for evaluating and maintaining legitimate circular dependencies. Enterprise architects must manually intervene to address complex dependency configurations during transformation activities [3].

Continuous verification and validation protocols remain primarily missing from current migration solutions and frameworks. Most platforms execute comprehensive validation only after completing transformation activities rather than during development iterations. Early error identification capabilities could prevent problematic transformations from spreading through complex migration processes. Real-time feedback protocols enable dynamic modification of transformation strategies based on intermediate validation outcomes. Behavioral equivalence evaluation requires a comprehensive understanding of system semantics that extends beyond surface-level code evaluation. Property-based verification methodologies can identify subtle behavioral variations that may not appear through traditional testing procedures. Integration with continuous development practices requires automated verification capabilities that function throughout iterative development activities. Current tools lack the advanced verification protocols necessary for maintaining system accuracy during complex transformation operations [3].

Shared state coordination across distributed agent boundaries introduces fundamental coordination obstacles for multi-agent migration platforms. Independent agents typically maintain separate perspectives of system state during concurrent processing operations. Information synchronization becomes essential for maintaining consistency across simultaneous transformations of interconnected system elements. Traditional message transmission methods introduce communication delays and potential information loss during agent interactions. Real-time collaboration demands sophisticated shared memory frameworks that can manage concurrent access from multiple specialized agents. Conflict resolution protocols must handle situations where agents simultaneously attempt to modify overlapping system resources or dependencies. Enterprise-scale complexity requires robust state coordination methodologies capable of managing massive code repositories with intricate dependency networks [4].

Positioning TriArchitect

Multi-agent system design emphasizes the significance of collaborative frameworks that enable sophisticated problem resolution through distributed intelligence capabilities. Symbiotic relationships among agents generate synergistic effects where combined capabilities surpass individual agent contributions. Shared knowledge frameworks eliminate information constraints that affect traditional sequential processing methodologies. Dynamic consensus protocols ensure that collaborative decisions satisfy multiple validation criteria before implementation activities. Real-time communication systems enable adaptive strategies that respond to emerging system requirements and operational constraints. Agent specialization maintains domain expertise while preventing the isolation challenges that impact independent processing platforms. The symbiotic methodology represents a fundamental transformation from coordination-based frameworks toward genuine collaborative intelligence [4].

Modern development processes, stressing iterative improvement and fast feedback systems, conflict with conventional batch-oriented migration approaches. Incremental processing allows for smooth integration with current toolchains for development and quality assurance efforts. Enterprise software development increasingly includes real-time processing capability requirements in continuous integration and deployment techniques. Real-time verification delivers immediate feedback about transformation quality and behavioral equivalence. Dynamic adjustment capabilities allow migration platforms to respond to changing requirements during active development activities. Integration compatibility with established enterprise toolchains becomes essential for widespread adoption across large organizations. Modern

development practices require migration methodologies that complement rather than disrupt existing processes [3].

Continuous validation protocols provide substantial advantages over traditional post-processing verification methodologies in enterprise transformation situations. Real-time consensus systems prevent problematic transformations from spreading through complex system frameworks and reaching production implementations. Multiple validation perspectives ensure a comprehensive evaluation of proposed changes from different technical and business viewpoints. Dynamic coordination enables specialized agents to collaborate effectively on complex transformation challenges that require diverse expertise. Consensus algorithms maintain safety standards while enabling automated processing at enterprise scale without compromising operational reliability. The methodology successfully balances automation benefits with risk management requirements that are essential for mission-critical business applications [4].

Architecture Type	Communication Pattern	Primary Limitation
Sequential Agent Chains	Predetermined handoff workflows	Context decay during transitions
Parallel Independent Agents	Isolated concurrent processing	Consistency challenges across outputs
TriArchitect Symbiotic	Continuous bidirectional collaboration	Requires sophisticated coordination protocols

Table I: Agent Architecture Comparison Framework. [3, 4]

3. TriArchitect Framework Architecture

Core Architectural Principles

Shared-state design transforms how distributed intelligent platforms handle collaborative problem resolution. Traditional multi-agent frameworks maintain separate knowledge databases that create communication barriers. Individual agents function with restricted awareness of system context and transformation goals. Shared-state platforms eliminate information divisions by creating unified data structures accessible to all agents. This design prevents vital information loss during agent communications. System-wide understanding develops from collective knowledge that surpasses individual agent capacities [5].

Continuous bidirectional communication enables dynamic collaboration among specialized processing units. Real-time data exchange supports flexible strategies addressing emerging challenges during transformation activities. Agents deliver immediate feedback regarding proposed modifications and their effects on system behavior. Dynamic consultation allows agents to request expertise from other specialists when facing complex situations. Bidirectional communication removes processing constraints common in traditional sequential frameworks [6].

Typed Migration Graph

Dynamic ontology frameworks deliver a comprehensive representation of software elements, including code parts, configuration items, and build systems. The ontological structure captures semantic connections extending beyond traditional dependency diagrams. Code elements preserve contextual details including business logic purpose, and architectural importance. Configuration parts maintain operational limits and deployment needs during transformation activities. The dynamic characteristics enable real-time modifications as transformation activities alter system structures [7].

Semantic relationship mapping documents the complete service interaction context within enterprise software frameworks. Traditional dependency evaluation focuses on syntactic connections like function calls among system parts. Semantic mapping expands analysis to include behavioral agreements and business rule implementation patterns. Cross-service communication configurations receive explicit representation, including data movement features and timing dependencies [6].

Three-Agent Specialization

The Archeologist Agent uses advanced pattern recognition methods to understand existing system structures and extract embedded business knowledge. Pattern detection examines system development records to distinguish stable architectural choices from temporary solutions. Code archaeology methods review version control records and structural modifications to understand the original system purposes. Business rule extraction combines static code evaluation with natural language processing of comments and documentation [5].

The Architect Agent concentrates on creating cloud-native target frameworks that optimize modern operational needs while respecting identified limits. Suitable for distributed computing scenarios, cloud-native design stresses scalability, fault tolerance, and operational efficiency. Constraint-aware transformation planning takes performance needs, integration compatibility, and resource limits into account. Multi-objective methods balancing several demands, including performance, cost, and maintainability, define target optimization [7].

The Validator Agent delivers comprehensive equivalence evaluation through multiple verification methods, including behavioral testing and property-based verification techniques. Abstract Syntax Tree analysis identifies structural modifications and behavioral implications with precision. Behavioral equivalence testing ensures transformed systems maintain essential features while enabling performance improvements. Continuous evaluation provides real-time feedback about transformation safety throughout migration activities [5].

Agent Type	Primary Function	Key Capability
Archeologist	Legacy system analysis	Business rule extraction from embedded code
Architect	Target design generation	Cloud-native architecture optimization under constraints
Validator	Equivalence verification	Abstract syntax tree differential analysis

Table II: Three-Agent Specialization Matrix. [5]

Cyclic Consensus Protocol

Signed migration verdict creation establishes formal approval procedures, ensuring comprehensive validation before applying transformations to production systems. Each agent provides explicit evaluations based on specialized domain knowledge and analysis outcomes. Digital signatures create auditable records supporting compliance needs and quality assurance activities. The verdict procedure considers multiple validation standards, including technical accuracy and business rule preservation [6]. Pre-merge verification gates integrate with existing development processes to provide automated safety checks before code integration. Verification gates examine proposed modifications against established architectural standards and business rule requirements. Integration with continuous systems enables automated blocking of problematic transformations before system stability impact [7].

4. Production Deployment and Results

Case Study Implementation

Multi-cloud migration methodologies provide essential bases for enterprise-scale changes across many different technological environments. Modern companies run distributed systems covering several cloud providers and on-premise infrastructure components. Methods of migration have to be flexible enough to handle elaborate networking configurations and uneven service capabilities across several cloud environments. Real-world case studies emphasize the need for thorough planning and staggered implementation plans. Enterprise environments present particular difficulties, including operational continuity demands [8] and legal compliance requirements.

Migration Approach	Manual Effort Reduction	Build Accuracy Rate
Manual Human-Only	Baseline reference point	Variable depending on expertise
Single-Agent Automated	Moderate reduction achieved	Inconsistent due to context limitations
TriArchitect Framework	Substantial reduction demonstrated	High consistency through consensus protocol

Table III: Performance Metrics Comparison. [8]

The deployment covered interconnected services supporting consumer electronics products requiring real-time performance features. These platforms demand extremely low latency responses and high availability to maintain acceptable user experience standards. Complex hardware interactions require specialized communication protocols and timing specifications that must be preserved during migration operations. Service evolution occurred through multiple development cycles with different architectural approaches and technology selections. Dependency relationships developed organically through iterative development, creating intricate interconnection configurations [9].

Phased rollout strategies reduce operational risks while enabling systematic validation of migration methods across different service types. Initial deployment phases target non-critical services exhibiting representative complexity patterns without affecting essential business functions. Pilot services provide opportunities to validate framework capabilities and identify potential problems before expanding deployment scope. Service selection standards emphasize dependency complexity, existing test coverage, and monitoring infrastructure sophistication [8].

Performance Analysis

Software migration performance assessment encompasses multiple aspects, including transformation precision, processing efficiency, and operational impact evaluation. Traditional migration methods require extensive manual effort involving multiple specialists and lengthy coordination operations across organizational teams. Automated frameworks significantly reduce the time and resources needed for successful system transformations. Performance gains stem from eliminating manual analysis operations that typically consume substantial portions of migration project schedules [9].

Operational enhancements manifest through improved system maintainability and reduced ongoing support requirements for migrated applications. Modern architectural patterns promote cleaner separation of concerns and more predictable system behaviors. Consistent coding conventions across migrated services simplify the development team onboarding and knowledge transfer activities. Improved resource utilization patterns emerge from systematic optimization during transformation operations. Legacy platforms often exhibit inefficient resource consumption patterns that developed incrementally over time [10].

Development productivity measurements reveal significant improvements in feature development cycles and maintenance operations following migration completion. Modernized codebases demonstrate reduced complexity in common development tasks, including debugging, testing, and feature addition activities. Automated testing coverage increases substantially through systematic test generation during migration operations. Build processes become more reliable through the elimination of legacy toolchain dependencies and the standardization of development environments [9].

Comparative Assessment

Database migration strategies across different industries reveal significant variations in approach effectiveness and implementation obstacles. Financial services organizations require specialized handling of transactional consistency and regulatory compliance during transformation activities. Healthcare institutions face unique constraints related to patient data privacy and system availability needs. Manufacturing companies must maintain operational continuity while transforming industrial control systems and production databases. Each industry presents distinct migration challenges requiring customized methods and specialized knowledge [10].

Cloud migration comparative studies demonstrate substantial differences between manual human-driven methods and automated transformation platforms. Human-only migrations provide thorough analysis and careful attention to business requirements, but require extensive time investments. Manual methods excel at handling complex edge cases and ambiguous transformation scenarios requiring expert judgment. However, human-driven processes struggle with scale and consistency when applied to large enterprise platforms [8].

Development velocity improvements become evident through systematic comparison of before and after transformation metrics across multiple organizational situations. Migrated systems demonstrate reduced complexity in common development activities, including feature implementation and bug resolution processes. Code review cycles become more efficient through consistent architectural patterns and improved system organization [9].

Safety and Risk Management

The best methods for multi-cloud migration underline how critical ongoing verification systems are throughout projects, transforming companies. Sophisticated monitoring and verification techniques are needed in business settings to find possible issues before they affect manufacturing processes. Risk reduction plans have to consider the complexity of distributed systems, including on-premises infrastructure and many cloud platforms. Automated verification solutions offer real-time monitoring of change activities, therefore helping to keep system integrity and behavioural equivalence [8].

Human escalation mechanisms activate automatically when automated systems encounter transformation scenarios requiring expert judgment and domain-specific knowledge. Security-critical code receives specialized handling protocols ensuring compliance with organizational security policies and regulatory requirements. Edge case management procedures address unusual scenarios falling outside standard transformation patterns and automated handling capabilities [9].

Implementation Insights

Critical success factors for enterprise-scale deployment include comprehensive initial codebase analysis, careful calibration of agent cooperation protocols, and establishment of clear escalation procedures. The most common failure modes involved edge cases where business rules were embedded in unexpected locations within the codebase. These scenarios highlighted the importance of sophisticated pattern recognition capabilities and comprehensive system analysis [10].

Mitigation strategies developed during deployment focused on proactive identification of challenging scenarios and preparation of specialized handling methods. Agent coordination mechanisms required fine-tuning based on specific organizational contexts and system characteristics. Performance optimization became necessary to handle the scale and complexity of enterprise codebases while maintaining interactive response times [8].

Industry Sector	Primary Challenge	Specialized Requirement
Financial Services	Transactional consistency maintenance	Regulatory compliance throughout transformation
Healthcare Systems	Patient data privacy protection	Continuous system availability demands
Manufacturing	Operational continuity preservation	Industrial control system compatibility

Table IV: Industry-Specific Migration Challenges. [10]

Conclusion

The successful deployment of TriArchitect demonstrates that shared-state multi-agent architectures can effectively resolve dependency hallucination problems that have limited automated migration tool effectiveness in enterprise environments. Through continuous collaboration between specialized agents mediated by a shared Typed Migration Graph, the framework achieves both the scale and safety required for mission-critical legacy modernization initiatives. The symbiotic agent communication patterns provide a template for addressing complex enterprise transformation challenges where context preservation and continuous verification are essential requirements. The Cyclic Consensus Protocol offers a systematic solution for ensuring transformation safety in automated systems across various domains. Production validation reveals that automated migration can achieve substantial efficiency improvements while maintaining operational reliability standards that enterprises require for business-critical systems. The framework's balance between automation and human oversight provides a practical path forward for risk-averse organizations seeking to modernize legacy infrastructure without compromising system integrity. Future directions include extending the shared-state architecture to other transformation domains such as database migration and API modernization, developing context window optimization techniques for larger codebases, and implementing enhanced detection algorithms for improved transformation reliability. The broader industry impact lies in demonstrating that artificial intelligence can successfully address high-stakes engineering problems while maintaining safety and reliability standards essential for enterprise operations. As organizations continue confronting legacy modernization challenges, frameworks like TriArchitect provide evidence that automated solutions can be both effective and trustworthy when properly architected, ultimately addressing technical debt burdens that constrain organizational innovation and competitiveness across the global economy.

References

- [1] Grand View Research, "Application Modernization Services Market (2024 - 2030), Size, Share & Trends Analysis Report By Service Type, By Deployment (Public Cloud, Private Cloud), By Enterprise Size (Large Enterprise, SMEs), By Industry Vertical, By Region, And Segment Forecasts," Grand View Research Market Report, 2023. [Online]. Available: <https://www.grandviewresearch.com/industry-analysis/application-modernization-services-market-report>
- [2] Vinicius et al., "Designing Multi-Agent Intelligence," Microsoft Developer Blog, 2024. [Online]. Available: <https://developer.microsoft.com/blog/designing-multi-agent-intelligence>
- [3] Gireesh Kambala, "Digital Transformation In IT: Challenges and Opportunities in Enterprise Systems," ResearchGate, 2025. [Online]. Available: https://www.researchgate.net/publication/388594673_Digital_Transformation_In_IT_Challenges_and_Opportunities_in_Enterprises_Systems
- [4] Junda He et al., "LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead," ACM Digital Library, 2025. [Online]. Available: <https://dl.acm.org/doi/10.1145/3712003>
- [5] Kshitij Bakliwal et al., "A Multi-Agent System architecture to implement Collaborative Learning for social industrial assets," ScienceDirect, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405896318315477>
- [6] Manal El Jaouhari et al., "Collaborative & Bi-directional Communication in Multi-Agent Metaheuristic Optimization," ResearchGate, 2025. [Online]. Available: https://www.researchgate.net/publication/397778409_Collaborative_Bi-directional_Communication_in_Multi-Agent_Metaheuristic_Optimization
- [7] Juhnnyoung Lee, Richard Thomas Goodwin, ResearchGate, "Ontology management for large-scale enterprise systems," ResearchGate, 2006. [Online]. Available: https://www.researchgate.net/publication/222436355_Ontology_management_for_large-scale_enterprise_systems
- [8] Obaloluwa Ogundairo, "Multi-Cloud Migration Best Practices for Enterprises: Tools, Challenges, and Real-World Case Studies for Seamless Transitions," ResearchGate, 2025. [Online]. Available: https://www.researchgate.net/publication/397903644_Multi-Cloud_Migration_Best_Practices_for_Enterprises_Tools_Challenges_and_Real-World_Case_Studies_for_Seamless_Transitions
- [9] Issam Al-Azzoni et al., "Performance evaluation for software migration," ACM Transactions on Software Engineering and Methodology, 2021. [Online]. Available: https://www.researchgate.net/publication/234820362_Performance_evaluation_for_software_migration
- [10] Journal of Systems and Software, "Comparative Analysis Of Cloud Migration Strategies Across Industries For Database Migration," ResearchGate, 2025. [Online]. Available: https://www.researchgate.net/publication/391284367_Comparative_Analysis_Of_Cloud_Migration_Strategies_Across_Industries_For_Database_Migration